École Normale Supérieure de l'Enseignement Technique (ENSET) — Mohammedia

Filière : Master Systèmes Distribués et Intelligence Artificielle (SDIA)

Année Universitaire 2026-2027

RAPPORT DE TEST TECHNIQUE

Systèmes Distribués — ADRIA / ENSET 2027

Plateforme bancaire micro-services : Spring Cloud, Feign/Resilience4J, Kafka, agent IA MCP et client Angular

Étudiant	Badr Eddine TOUBANI
Professeur	Pr. Mohamed YOUSSFI
Filière	Master SDIA
Établissement	ENSET Mohammedia
Entreprise	ADRIA Business & Technology
Dépôt	https://github.com/toubani12/TEST-ADRIA-ENSET
Date	30/10/2026

Tableau 0.1 — Informations du projet

SOMMAIRE

Introduction Générale 4
Étape 1 : Architecture et socle Spring Cloud 4
1.1 Vue d'ensemble 4
1.2 Stack technique 5
1.3 Organisation en couches 5
Étape 2 : Services métier et communication synchrone 5
2.1 account-service et transaction-service 5
2.2 Règles métier 5
2.3 OpenFeign et Resilience4J 6
Étape 3 : Communication asynchrone avec Kafka 6
3.1 Publication après commit 6
3.2 audit-service 6
3.3 Résultat 6
Étape 4 : Chatbot agentique 7
4.1 Principe 7
4.2 Problèmes résolus 7
4.3 Tests réels 7
Étape 5 : Client Angular 8
Étape 6 : Sécurité, tests et Git 10
6.1 Sécurité Keycloak 10
6.2 Tests 10
6.3 Historique Git 10
Conclusion et Perspectives 11
7.1 Bilan du Projet 11
7.2 Limites et Perspectives 11
7.3 Recommandations Opérationnelles 11
Références Bibliographiques 12

LISTE DES ABRÉVIATIONS

Abréviation	Signification
API	Application Programming Interface
CB	Circuit Breaker (disjoncteur Resilience4J)
DTO	Data Transfer Object
JWT	JSON Web Token
KRaft	Kafka Raft (mode sans Zookeeper)
LLM	Large Language Model
MCP	Model Context Protocol
REST	Representational State Transfer
SSE	Server-Sent Events

Tableau 0.2 — Abréviations utilisées

Introduction Générale

L'épreuve demande de concevoir une plateforme bancaire distribuée : gestion des comptes, des transactions (paiement, dépôt, virement), communication synchrone tolérante aux pannes, communication asynchrone par événements, agent conversationnel capable d'exécuter des opérations bancaires, et client web. Le projet a été réalisé dans le dépôt test-adria-backend, avec un commit par fonctionnalité.

Le rapport suit la démarche de réalisation :

1. Architecture et socle Spring Cloud (configuration, découverte, passerelle).
2. Micro-services métier account-service et transaction-service (REST, GraphQL, MCP) et communication Feign/Resilience4J.
3. Communication asynchrone Kafka et service d'audit, puis agent IA et client Angular.
4. Sécurité Keycloak, tests, bilan et limites.

Étape 1 : Architecture et socle Spring Cloud

1.1 Vue d'ensemble

Chaque responsabilité est portée par un micro-service Spring Boot indépendant, enregistré dans Eureka et configuré par le Config-service. Le front et les clients externes passent par la passerelle ; le chatbot dialogue avec les services métier via leurs serveurs MCP.

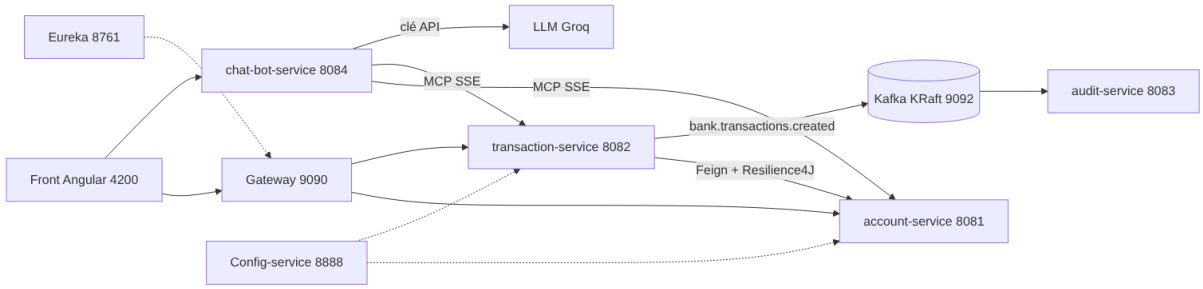


Figure 1.1 — Architecture de la plateforme

Service	Port	Rôle
Config-service	8888	Spring Cloud Config (native), partage bank.transaction.max-amount=1000
discovery-service	8761	Annuaire Eureka
gatway-service	9090	Spring Cloud Gateway (webmvc), routes lb:// vers les services
account-service	8081	Comptes : création, solde, débit/crédit, blocage
transaction-service	8082	Paiements, dépôts, virements ; producteur Kafka
audit-service	8083	Consumer Kafka, journal d'audit
chat-bot-service	8084	Agent IA : LLM + tools MCP
Front Angular	4200	Client web
Kafka (KRaft)	9092	Broker d'événements, sans Zookeeper

Tableau 1.1 — Services et ports

1.2 Stack technique

Élément	Version / choix
Spring Boot	4.0.9
Spring Cloud	2025.1.4
Spring AI	2.0.1 (MCP server webmvc, MCP client, OpenAI)
Java	21
Base de données	H2 en mémoire par service
Messagerie	Apache Kafka 3.9.1 (KRaft, docker-compose)
Front	Angular 22 (composants standalone, signals)
Sécurité	Keycloak 26.2, OAuth2 Resource Server JWT

Tableau 1.2 — Stack technique

1.3 Organisation en couches

Chaque service métier respecte la séparation imposée par l'énoncé. Les trois points d'entrée (REST, GraphQL, MCP) ne contiennent aucune logique métier et délèguent tous à la même méthode de la couche service.

```
domain → repository → service → web/rest | web/graphql | mcp
```

Le POM racine est un POM agrégateur qui déclare les sept modules, ce qui permet à IntelliJ et à Maven de détecter tous les services en un seul projet.

Étape 2 : Services métier et communication synchrone

2.1 account-service et transaction-service

account-service gère les comptes (référence client, devise ISO, solde, statut ACTIVE/BLOCKED). transaction-service orchestre les paiements, dépôts et virements en appelant account-service. Les deux services exposent la même logique en REST, en GraphQL (GraphiQL activé) et sous forme de tools MCP.

2.2 Règles métier

Règle	Comportement
Plafond par opération	Paramètre partagé par le Config-service (1000). Un paiement de 5000 renvoie 422 : « Le montant 5000 dépasse le plafond autorisé de 1000 ».
Montant	Strictement positif, sinon erreur métier.
Devise	La devise de l'opération doit être celle du compte (CurrencyMismatchException).
Compte bloqué	Toute opération sur un compte non actif est refusée (AccountBlockedException).
Solde insuffisant	Paiement et virement vérifient le solde (InsufficientBalanceException).

Règle	Comportement
Virement	Deux écritures TRANSFER_OUT / TRANSFER_IN liées par transferId ; si le crédit échoue, le débit est compensé.

Tableau 2.1 — Règles métier implémentées

Les erreurs sont renvoyées au format ProblemDetail par un @RestControllerAdvice (404, 422 pour une règle métier, 503 si account-service est indisponible).

2.3 OpenFeign et Resilience4J

transaction-service appelle account-service via un client OpenFeign résolu par Eureka. Le circuit breaker Resilience4J est activé sur le client Feign avec une FallbackFactory : une erreur fonctionnelle (4xx, par exemple compte introuvable) est propagée telle quelle, tandis qu'une panne technique (timeout, service arrêté, circuit ouvert) devient une AccountServiceUnavailableException traduite en 503. Aucun solde n'est inventé : l'opération est refusée.

Paramètre	Valeur
Fenêtre glissante	10 appels (minimum 5)
Seuil d'ouverture	50 % d'échecs
Durée circuit ouvert	10 s, puis 2 appels en semi-ouvert
Time limiter	3 s
Exceptions ignorées	NotFound, BadRequest, Conflict, UnprocessableEntity

Tableau 2.2 — Configuration Resilience4J

Étape 3 : Communication asynchrone avec Kafka

3.1 Publication après commit

Après la persistance d'une transaction, le service publie un événement Spring applicatif. Un listener @TransactionalEventListener(phase = AFTER_COMMIT) l'envoie dans le topic bank.transactions.created (3 partitions) : si la transaction JPA est annulée, aucun événement n'est publié. La clé du message est accountId, ce qui garantit l'ordre des événements d'un même compte dans une partition.

```
{"eventId","eventType":"TransactionCreated","transactionId","accountId","amo
                        unt","currency","occurredAt"}
```

3.2 audit-service

audit-service consomme le topic (groupe audit-service), journalise chaque événement et le stocke dans sa propre base, sans jamais modifier les données bancaires. L'endpoint GET /api/audits expose les 100 derniers événements.

3.3 Résultat

Test réel : un paiement de 250 EUR sur CUST-001 via la passerelle a produit un événement TransactionCreated reçu par audit-service sur la partition 1, offset 0, avec la clé égale à l'accountId du compte.

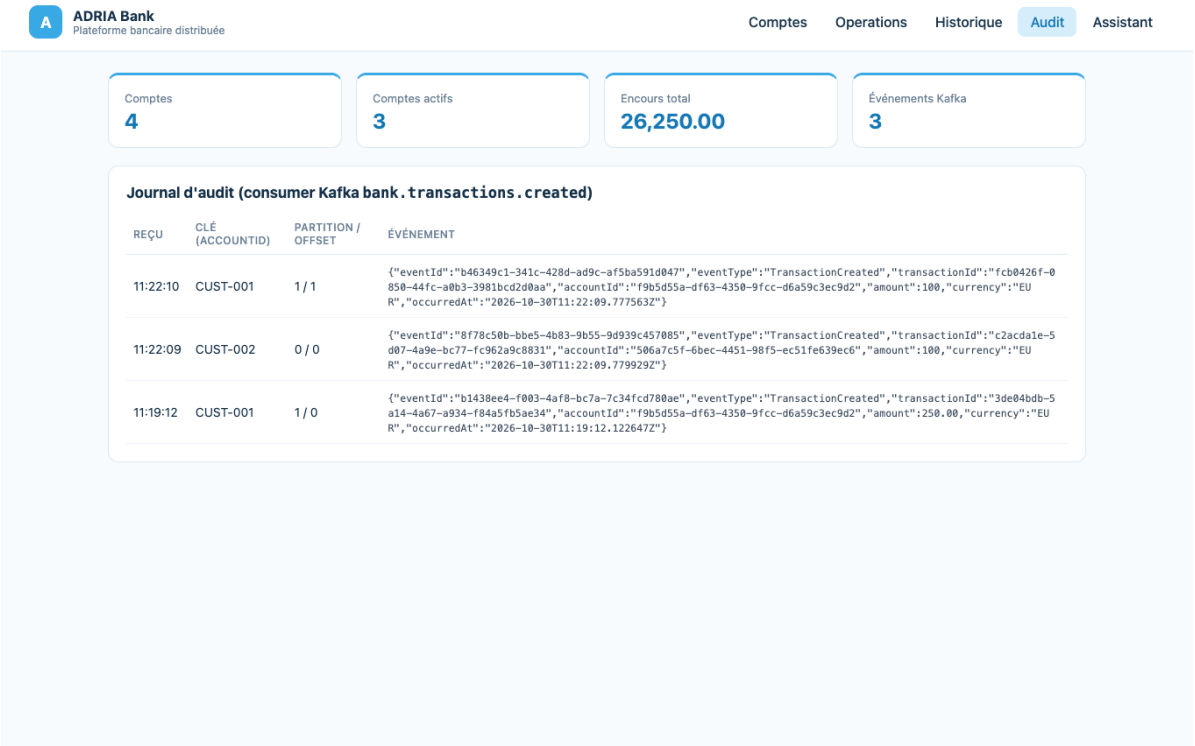


Figure 3.1 — Journal d'audit alimenté par Kafka

Étape 4 : Chatbot agentique

4.1 Principe

L'agent n'utilise aucun modèle entraîné localement : il appelle un LLM distant (Groq, API compatible OpenAI) avec une simple clé d'API lue depuis .env (non versionné). Spring AI ChatClient lui fournit les tools du client MCP (transport SSE) connecté à account-service et transaction-service : 12 tools MCP sont chargés au démarrage. Le prompt système lui interdit d'inventer un solde ou une transaction.

4.2 Problèmes résolus

- Spring AI 2 utilise le SDK officiel OpenAI : l'URL de base doit se terminer par /v1 (sinon 404 sur /openai/chat/completions).
- openai/gpt-oss-120b a été rejeté par Groq lors du tour suivant l'appel de tool (propriété reasoning_content non supportée) ; le modèle retenu est qwen/qwen3.8-27b.

4.3 Tests réels

Question	Réponse de l'agent
Quel est le solde de CUST-001 ?	Le solde du compte CUST-001 est de 4 750,00 EUR (statut : actif).

Question	Réponse de l'agent
Fais un virement de 100 EUR de CUST-001 vers CUST-002, puis donne les deux soldes.	Virement effectué ; CUST-001 : 4 650,00 EUR, CUST-002 : 1 300,00 EUR.

Tableau 4.1 — Échanges réels avec l'agent (endpoint POST /api/chat)

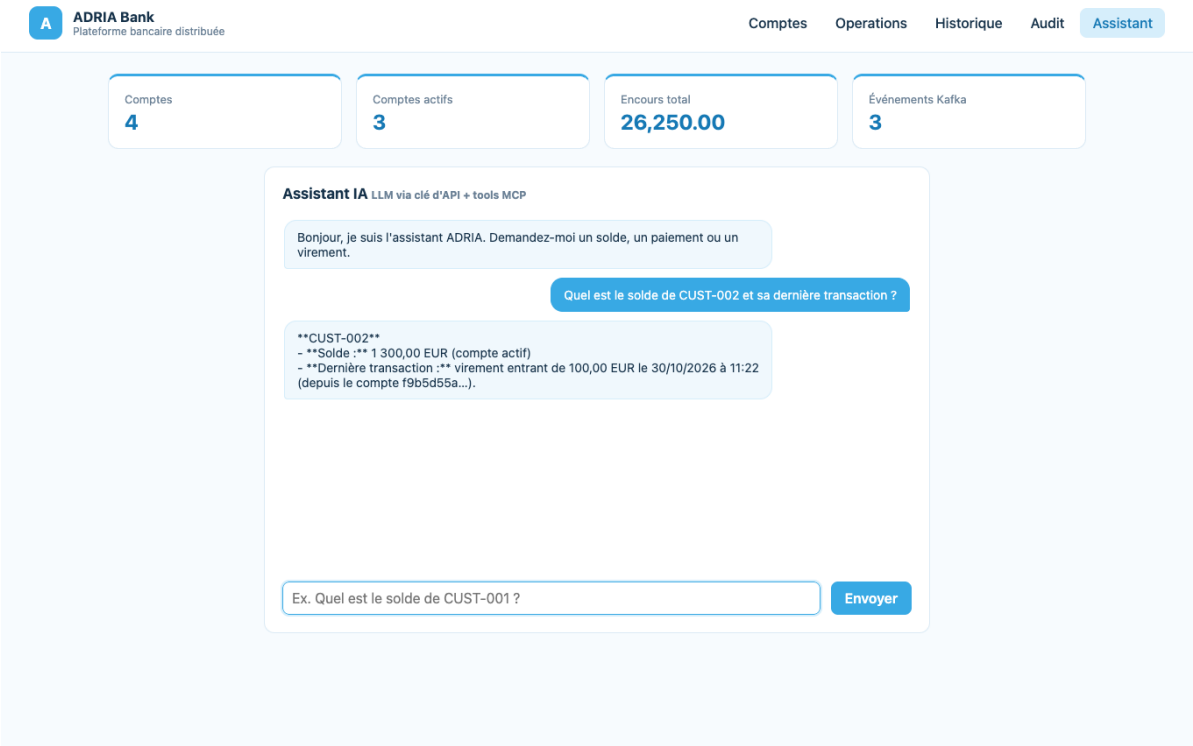


Figure 4.1 — Assistant IA dans le client Angular

Étape 5 : Client Angular

Le client Angular 22 (thème blanc et bleu ciel) propose cinq onglets : Comptes (liste, création, blocage/activation), Opérations (paiement, dépôt, virement), Historique (filtrable par compte), Audit (événements Kafka) et Assistant. En développement, un proxy redirige /api/accounts et /api/transactions vers la passerelle, /api/audits vers audit-service et /api/chat vers le chatbot.

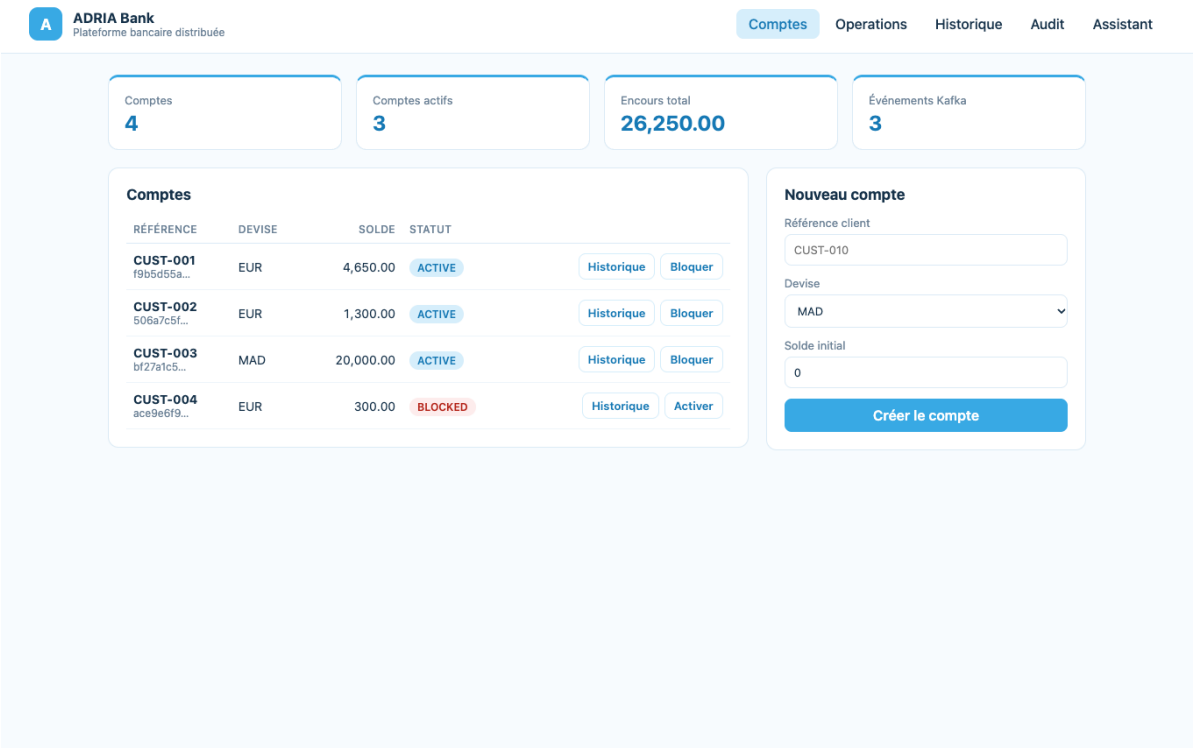


Figure 5.1 — Tableau de bord des comptes

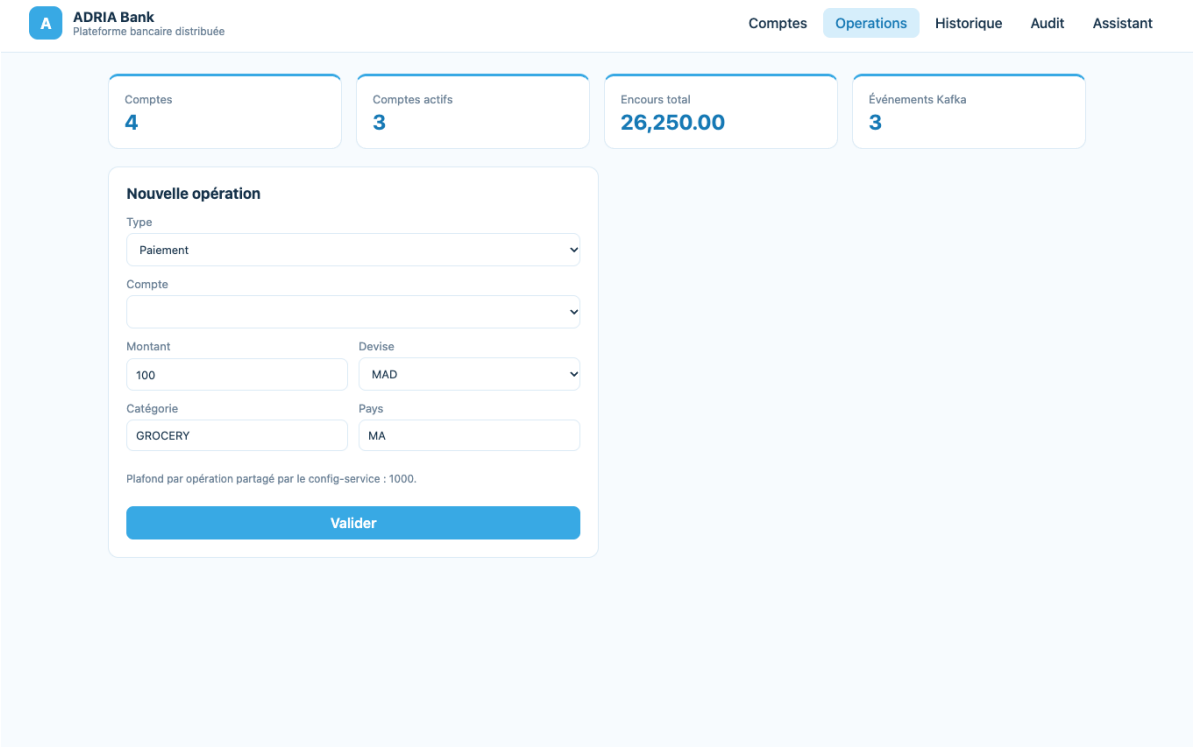


Figure 5.2 — Formulaire d'opération

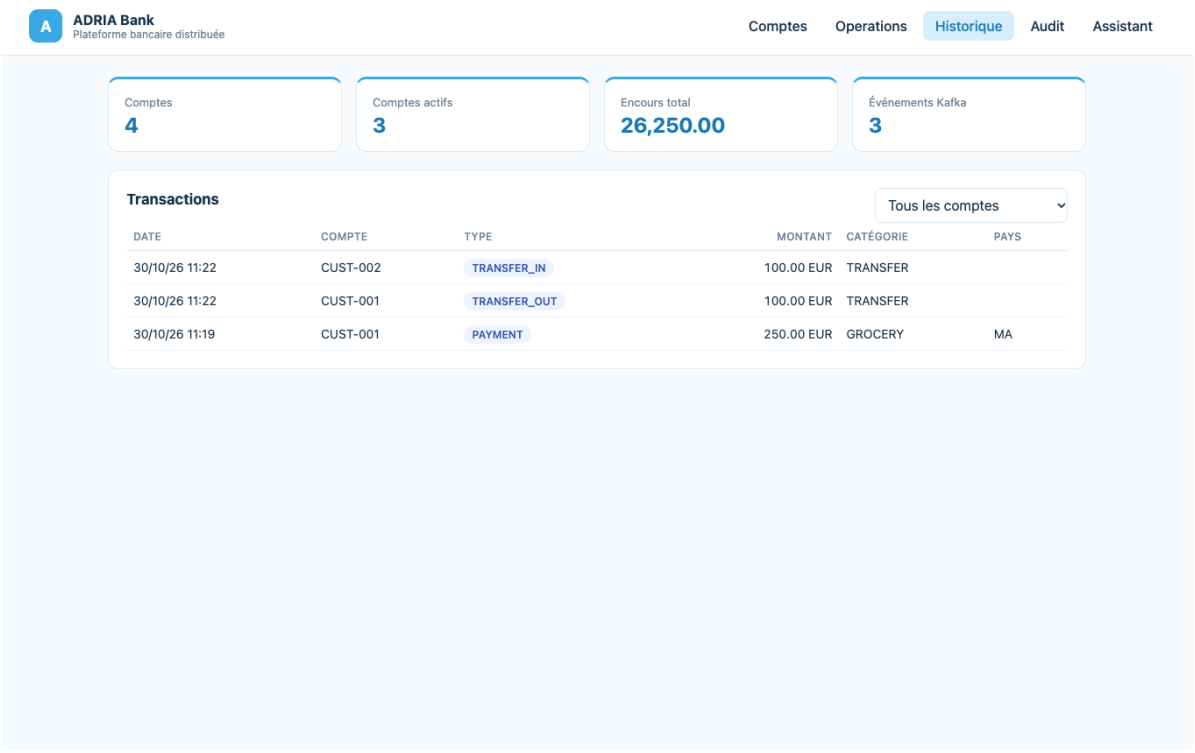


Figure 5.3 — Historique des transactions

Étape 6 : Sécurité, tests et Git

6.1 Sécurité Keycloak

La sécurité a été ajoutée en dernier, conformément à la démarche choisie (fonctionnel d'abord) : Keycloak 26.2 dans docker-compose, realm bank-realm importé depuis keycloak/bank-realm-realm.json, client public bank-front pour le front, rôles USER et ADMIN. Les services sont configurés en OAuth2 Resource Server JWT, activable via la propriété app.security.enabled.

6.2 Tests

Module	Tests	Résultat
account-service	19	OK
transaction-service	12	OK
audit-service	1	OK
Config-service	1	OK
discovery-service	1	OK
gatway-service	1	OK
Total (reactor Maven, 7 modules)	34	BUILD SUCCESS

Tableau 6.1 — Résultats de mvn test

6.3 Historique Git

Un commit a été fait par fonctionnalité :

- POM agrégateur pour la détection des modules par IntelliJ.
- Publication Kafka après commit dans transaction-service.
- audit-service (consumer Kafka).
- chat-bot-service (LLM via clé d'API + tools MCP).
- Client Angular.
- Circuit breaker Resilience4J et fallback Feign.
- docker-compose avec Kafka KRaft.
- Correctifs du chatbot (URL /v1, modèle) et scripts de lancement.
- Sécurité Keycloak.

Conclusion et Perspectives

7.1 Bilan du Projet

1. Socle Spring Cloud complet : configuration centralisée, découverte, passerelle.
2. Deux services métier exposant la même logique en REST, GraphQL et MCP, avec règles métier testées.
3. Communication synchrone résiliente (Feign + Resilience4J) et asynchrone (Kafka + audit).
4. Agent IA opérationnel sur données réelles, client Angular et sécurité Keycloak.

7.2 Limites et Perspectives

- Partie H (détection de fraude) non réalisée faute de temps, la consigne étant de ne pas entraîner de modèle pendant l'épreuve. La démarche prévue : séparation stratifiée train/test, pipeline de prétraitement, seuil de décision choisi uniquement sur le train, évaluation par précision, rappel, F1 et matrice de confusion (classes déséquilibrées).
- Bases H2 en mémoire : passer à PostgreSQL pour la persistance.
- Conteneurisation des micro-services et pipeline CI/CD à compléter (seuls Kafka et Keycloak sont dans docker-compose).

7.3 Recommandations Opérationnelles

Aspect	Recommandation
Démarrage	Lancer docker-compose puis scripts/start-all.sh (ordre : config, discovery, services, gateway, chatbot).
Clé LLM	À placer dans .env (jamais dans Git) ; modèle configurable via LLM_MODEL.
Chatbot	Redémarrer le chatbot si un service MCP redémarre (connexion MCP établie au démarrage).

Tableau 7.1 — Recommandations

Références Bibliographiques

[1] Spring Cloud — documentation de référence (Gateway, OpenFeign, Config, Netflix Eureka).

[2] Resilience4J — documentation CircuitBreaker et TimeLimiter.

[3] Apache Kafka 3.9 — documentation, mode KRaft.

[4] Spring AI 2.0 — ChatClient, MCP Server et MCP Client.

[5] Keycloak 26 — Server Administration Guide.